

---

# DEEP LEARNING FOR VISUAL RECOGNITION

---

Department of Computer Science  
Aarhus University

Department of Computer Science  
Aarhus University

December 5, 2025

## ABSTRACT

This report details an image classification project using the Food-101 dataset [1], intentionally scoped down to simulate a data-limited setting (20 classes, 20 images per class). An initial model, utilizing a frozen ResNet18 encoder via transfer learning, quickly overfit the small dataset, achieving 57% validation accuracy. We systematically applied key regularization techniques such as Early Stopping, Dropout, and Data Augmentation. The subsequent experimental phase involved fine-tuning by unfreezing additional layers to leverage the encoder's representational power. We demonstrate how a combination of regularization and fine-tuning can be used to achieve higher performance and generalization capability in data scarce scenarios.

**Keywords** Deep learning · Transfer learning · Food-101 · Overfitting · Regularization

## 1 Introduction

The objective of this project is to train and evaluate an image classification model using transfer learning on a subset of the Food-101 dataset.

Food recognition models can be used in numerous domains such as healthcare and nutrition by enabling automatic meal logging and nutrient estimation for conditions like diabetes, cardiovascular disease, and obesity. Furthermore, we can imagine a use case where retail stores may benefit from faster checkout through automatic identification of produce and prepared foods. Waste-reduction management can also track and analyze discarded items to improve purchasing and portioning.

Our primary goal is to understand and implement effective mitigation strategies for overfitting, which is a common challenge when working with limited training data in deep learning.

Rather than building a convolutional neural network from scratch, we leverage transfer learning with a pre-trained ResNet18 model, which provides a strong foundation of visual features learned from large-scale image datasets. This approach is particularly relevant in real-world scenarios where collecting large labeled datasets is often very expensive, time-consuming, or impractical, such as in medical imaging, industrial quality control, or specialized domains with rare visual categories.

To create a realistic and challenging learning, we intentionally restricted our dataset to 20 classes with only 20 images per class, simulating conditions where data acquisition is severely limited. Initial experiments with the full dataset (1000 images per class) achieved very high accuracy (90%), which, while successful, provided limited opportunity to explore regularization techniques and their impact on generalization. By constraining the data, we create a controlled setting where overfitting emerges as a central problem, allowing us to systematically evaluate how different regularization and fine-tuning strategies affect model performance.

## 2 Related work

The initial performance ceiling for the Food-101 dataset was established by Bossard et al. [2], who used Random Forests (RF) to mine discriminative components and achieved an average accuracy of 50.76% on the 101 classes.

Bossard et al. noted that only CNNs were capable of substantially outperforming their robust RF approach [2]. Further specialized research in food recognition includes approaches like Wide-Slice Residual Networks [3], while generalized solutions for data scarcity often leverage Few-Shot Learning techniques such as Prototypical Networks [4].

In contrast to these earlier methods, our approach avoids both handcrafted part-based pipelines and highly specialized food-recognition architectures. Instead, we use a general-purpose ResNet-18 backbone with transfer learning and focus on improving performance in a low-data setting through systematic regularization (early stopping, dropout, data augmentation) and selective fine-tuning of deeper layers. Unlike few-shot methods that rely on metric learning or episodic training, our method stays within the standard supervised learning paradigm while remaining lightweight, model-agnostic, and easy to scale.

The foundation of our specific project is the Residual Network (ResNet) architecture, introduced by He et al. [5]. While the original paper showed that the deepest models like ResNet-152 achieved the lowest error on the massive ImageNet dataset, we opted for the much shallower ResNet-18. This decision was made under the assumption that our project operates in a data-constrained setting, the high capacity of deeper models would increase the likelihood of the model memorizing our dataset, and result in severe overfitting. Given that even the smaller ResNet-18 overfit almost immediately, we hypothesized that a much larger model, such as ResNet-152, would overfit even more severely. Its higher number of parameters would make overfitting much harder to control with standard regularization strategies. We proceeded with this assumption without conducting an explicit experiment on the deepest models.

### 3 Methods

#### 3.1 Data set

The Food-101 dataset [1] contains 101 food categories, each with 1,000 images, for a total of 101,000 images. To simulate a low-data scenario, we created a smaller subset (see Figure 1 for examples):

- Dataset size: 20 classes (e.g., pizza, hamburger, sushi, apple pie, etc.).
- Images per class: 20 images per class.
- Total training images: 400 images (40 Validation images).
- Test set: 400 images.



(a)



(b)



(c)

Figure 1: Examples of Food-101 images

#### 3.2 Network architecture

The architecture is based on transfer learning, using ResNet-18 as the feature extractor (encoder), pre-trained on ImageNet. ResNet-18 is composed of several types of layers that process images hierarchically:

- **Convolutional layers:** Extract low- and mid-level features such as edges, textures, and patterns.
- **Batch normalization layers:** Normalize the activations after each convolution to improve training stability and convergence.
- **ReLU activations:** Introduce non-linearity, enabling the network to learn complex patterns.

- **Residual blocks:** Combine multiple convolutional layers with skip connections, allowing gradients to flow more easily and mitigating the vanishing gradient problem in deeper networks.
- **Global average pooling:** Aggregates spatial features into a single vector per channel, reducing the number of parameters before the classifier.
- **Fully connected (FC) layer:** Maps the aggregated features to the class scores; in our case, it is adapted to output probabilities for 20 food classes.

Initially, only the final FC layer was unfrozen and trained, while all convolutional layers were kept frozen to preserve the general visual features learned from ImageNet. This setup allowed us to clearly observe overfitting and systematically measure the incremental effects of each mitigation strategy.

Following the initial experiments, we explored selective fine-tuning by progressively unfreezing the residual blocks, starting from the deepest layers (layer4, then layer3, layer2, and finally layer1). Our experiments showed that unfreezing three of the four blocks (layer4, layer3, and layer2) while keeping layer1 frozen yielded the best performance. Fully unfreezing all blocks led to overfitting, likely due to the limited size of our dataset. These results suggest that, in this low-data setting, preserving the early layers helps maintain general low-level features, while allowing higher-level features to adapt to the specific characteristics of the Food-101 subset.

### 3.3 Preprocessing

Before the images could be passed to the network, we applied standard preprocessing steps required for a pre-trained model like ResNet18. First, we normalized all the images. Concretely, we adjusted pixel values using the mean and standard deviation statistics associated with the ImageNet dataset. This aligns the Food-101 images with the distribution that ResNet18 was originally trained on. Second, we ensured that every image had the correct spatial resolution. All images were resized and cropped to  $224 \times 224$  pixels, which is the required input size for ResNet18.

### 3.4 Hyperparameter

We experimented with a range of hyperparameters to optimize model performance during our experiments. For batch size, we tested 2, 4, 16, and 64. Batch size of 4 produced the best results, while 64 performed poorly, likely due to the small dataset not supporting large-batch training. Training for 10, 50, and 100 epochs showed that performance flattened after about 20 epochs, indicating that the model quickly exhausted learnable patterns given the limited data.

#### 3.4.1 Dropout

To combat overfitting, we first introduced dropout in the classifier head. Dropout works by randomly deactivating a fraction of neurons during training, preventing the model from relying too heavily on a few highly tuned activations and encouraging it to learn more robust, distributed representations. We experimented with different dropout rates, but ended up using a dropout rate of  $p = 0.5$ , which provided strong regularization to counter the overfitting observed in the baseline while still maintaining enough capacity to learn from the limited data.

#### 3.4.2 Data augmentation

To further improve accuracy and address the remaining overfitting, we then incorporated data augmentation. We began with an aggressive augmentation setup, which included RandomErasing and strong geometric and color transformations. This approach increased generalization, although the augmentation led to reduced accuracy. In practice, combining the dropout with a less aggressive augmentation provided effective regularization while still allowing the model to learn useful class-specific patterns from the small dataset.

Figure 2 shows examples of the augmented images used during training, illustrating the transformations applied to increase dataset variability.



Figure 2: Result of making data augmentation

### 3.4.3 Learning rate

We tested several learning rates to understand how they affect performance. For training the decoder, a learning rate of 0.0001 resulted in poor accuracy, while 0.001 gave the best results. Increasing it to 0.01 caused the accuracy to drop, as the updates became too unstable.

During fine-tuning we used different learning rates for the encoder and decoder. The encoder performed best with a very small learning rate of 0.0001, which allowed gentle adjustments without damaging the pretrained features. The decoder, however, still benefited from the slightly larger 0.001, enabling it to adapt more quickly to the new task.

### 3.5 Optimizer

We used stochastic gradient descent (SGD) as the optimizer for all experiments, as it generally provides stable convergence for small datasets. Momentum was kept at standard values. We also experimented with adding weight decay, but it did not lead to noticeable improvements, since dropout already had a great effect on the overfitting problem.

## 4 Experiments and Results

We designed an iterative experimental sequence to isolate the impact of each mitigation strategy. In this section, we document the model’s behavior during training and report the final performance metrics achieved on the test set. We analyze how different regularization techniques affected the model’s ability to learn and generalize. We found that while regularization was essential to control severe overfitting in our low-data setting, fine-tuning the ResNet encoder delivered the largest performance boost and achieved the best overall accuracy.

### 4.1 Performance Summary Across Configurations

Table 3 summarizes each model’s performance on the test set of 400 images. The fine-tuned model from Run 6 achieved the best Top-1 accuracy.

Table 1: Summary of Test Set Performance Metrics for Each Model Configuration

| Configuration          | Acc. (Top-1)  | Err. (Top-1)  | Acc. (Top-5)  | Err. (Top-5)  |
|------------------------|---------------|---------------|---------------|---------------|
| Early Stopping (Run 2) | 0.5550        | 0.4450        | 0.8800        | 0.1200        |
| Dropout (Run 3)        | 0.5350        | 0.4650        | 0.8725        | 0.1275        |
| Data Aug. (Run 5)      | 0.5625        | 0.4375        | 0.8625        | 0.1375        |
| Fine-tuning (Run 6)    | <b>0.6250</b> | <b>0.3750</b> | <b>0.8975</b> | <b>0.1025</b> |

Figure 3: Performance metrics on the test set for each model configuration. Top-1, Top-5 accuracy and error are shown.

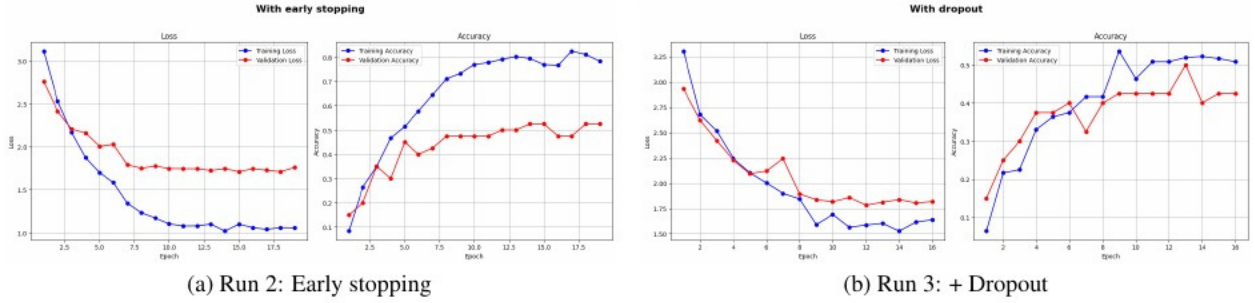
### 4.2 Run 1: Baseline

The baseline experiment trained the ResNet18 model with the encoder frozen and only the final FC layer trained, without any explicit regularization beyond the basic setup. With almost no regularization, the model overfit very quickly. Training accuracy reached around 80%, but validation accuracy stayed around 50%. This showed that the model could memorize the small training set without learning patterns that transfer well to new images. This run confirmed that overfitting is a real problem in our low-data setup and that regularization is needed to combat this.

### 4.3 Run 2: Early Stopping

We implemented early stopping to make training more efficient. Validation loss was monitored, and training was halted if the loss failed to decrease for a predefined number of epochs (patience). When triggered, the model reverted to the weights from the epoch with the lowest validation loss. Training accuracy stabilized around 80%, while validation accuracy remained lower, so overfitting persisted. Early stopping mainly improved training efficiency rather than reducing overfitting, and this run achieved a Test Set Top-1 Accuracy of **0.5550**.

Figure 4: Training and validation performance for early stopping and dropout strategies.



#### 4.4 Run 3: Dropout

In this experiment, we applied dropout ( $p = 0.5$ ) to the classifier head, as described in Section 3. Compared to the baseline, dropout reduced the gap between training and validation accuracy, which had been around 30 percentage points, indicating a clear reduction in overfitting. However, the overall test performance (Test Set Top-1 Accuracy of 0.5350) was slightly worse than with early stopping alone.

Our interpretation is that with so little data, dropout on top of a frozen encoder acts as a relatively strong regularizer and can remove too much useful signal when it is not supported by additional data or augmentation. So dropout did help to control the overfitting, but in this configuration it also made it harder for the model to learn enough from the few available examples.

Based on the relatively low accuracy of the model, we wanted to investigate what causes this. To see if any one of the classes were causing it we used a confusion matrix. This revealed that the model struggled with some classes especially grilled salmon, fried rice, waffles, gyoza, gnocchi, omelet, and hummus.

Using the confusion matrix we observed, for example, that 13 out of 20 images of fried rice were incorrectly predicted as macaroni and cheese. An example of this confusion is shown in Fig. 5(a).

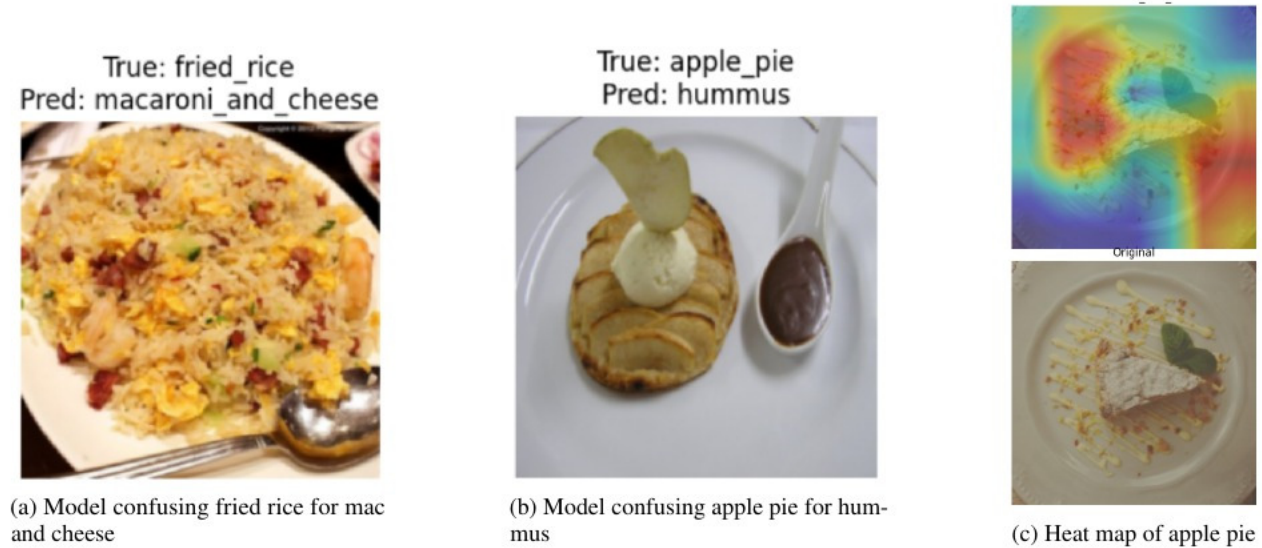


Figure 5: Examples of model confusion on different dishes.

Some misclassifications were easy to understand, such as confusing fried rice with macaroni and cheese, but others were less obvious. For example, in Fig. 5(b) the model misclassified an apple pie, and it was unclear what features led to this error. To investigate, we generated Grad-CAM heatmaps to visualize the regions of the images that the model focused on. These heatmaps allowed us to assess whether the network was attending to relevant parts of the image or to misleading visual cues.

The Grad-CAM visualizations showed that, in some cases, the model focused on less relevant regions, such as the glaze on the plate rather than the apple pie itself. This likely contributed to certain misclassifications, as the network relied on cues that can vary between images. Improving performance in these cases could involve providing more diverse training data, using targeted data augmentations to emphasize the main object, or incorporating attention mechanisms or segmentation to guide the model's focus.

#### 4.5 Run 4: Weight Decay

We added weight decay (L2 regularization) alongside dropout to further reduce overfitting. Dropout limits the model's reliance on specific neurons, while weight decay directly discourages large weight values by adding an L2 penalty to the loss. Large weights can make the network highly sensitive to small variations in the input, causing it to fit noise in the training data rather than general patterns. By keeping weights smaller, weight decay effectively smooths the model's mapping from inputs to outputs, making it less likely to memorize the limited training set and improving generalization to new images.

However, we still saw a roughly 10% gap between training and validation accuracy, showing that weight decay did not noticeably reduce overfitting. We believe the main reason is simply the limited dataset. With so few examples, the model quickly reached its capacity, so performance was more limited by data scarcity than by the size of the weights. Since the ResNet encoder was frozen, weight decay only affected the small fully connected layer at the end. The model's generalization mostly depended on the fixed ResNet features, which the L2 penalty couldn't improve, making its effect negligible.

#### 4.6 Run 5: Data Augmentation

We applied the augmentation strategies from Section 3 to increase training variability. Aggressive augmentation reduced the training-validation gap but slightly lowered Top-1 accuracy, while light augmentation offered a better balance, achieving 0.5625 Top-1 accuracy and improving generalization in our low-data scenario.

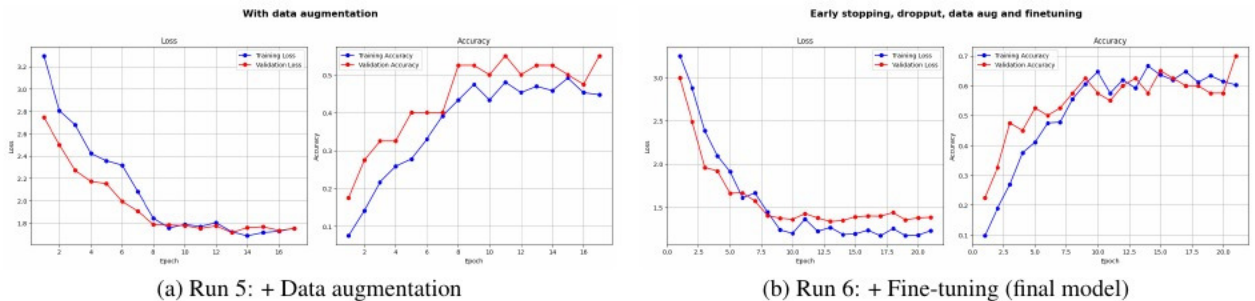
**Example (aggressive) training transform:**

```
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2,
                           contrast=0.2,
                           saturation=0.2,
                           hue=0.1),
    transforms.RandomGrayscale(0.2),
    transforms.ToTensor(),
    transforms.RandomErasing(0.3),
    transforms.Normalize(
        mean=[0.5, 0.5, 0.5],
        std=[0.5, 0.5, 0.5])
])
```

**Example (light) training transform:**

```
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.5, 0.5, 0.5],
        std=[0.5, 0.5, 0.5])
])
```

Figure 6: Training and validation performance for data augmentation and fine-tuning strategies.



#### 4.7 Run 6: Fine-Tuning

Following the regularization and data augmentation steps, we performed fine-tuning as described in Section 3. We unfroze the final convolutional block (Layer 4) of ResNet18 and trained it with a low learning rate, applying all established regularization methods. Gradually unfreezing the deeper residual blocks allowed the model to adapt its high-level features to the Food-101 subset, enabling it to focus on subtle, domain-specific visual cues, such as texture and ingredients, rather than relying solely on generic ImageNet features.

This fine-tuning step led to the best performance, with a Test Set Top-1 Accuracy of **0.6250**, which is a 7 percentage point improvement over Run 2, while the Test Set Top-5 Accuracy was **0.8975**. The gap between training and validation performance also reduced, indicating improved generalization.

A comparison of the confusion matrices for the regularized model (Run 3, Fig. 7) and the fine-tuned model (Run 6, Fig. 8) confirms the improvement achieved through fine-tuning. The number of correct classifications increased from 201 to 250 out of 400 test images, with notable improvements in specific classes, for instance, correct predictions for gyoza rose from 5 out of 20 to 17 out of 20.

Despite these gains, the model continued to struggle with certain classes like hummus and gnocchi. But overall the fine-tuning improved performance across almost all classes, though some challenging classes remained difficult.

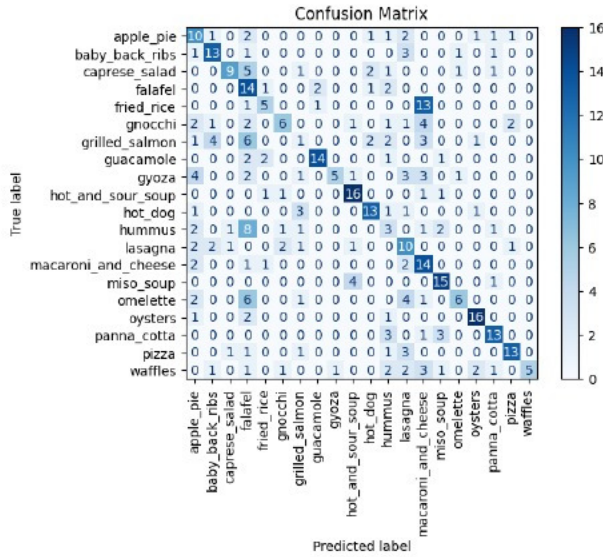


Figure 7: Run 3: Dropout model confusion matrix.

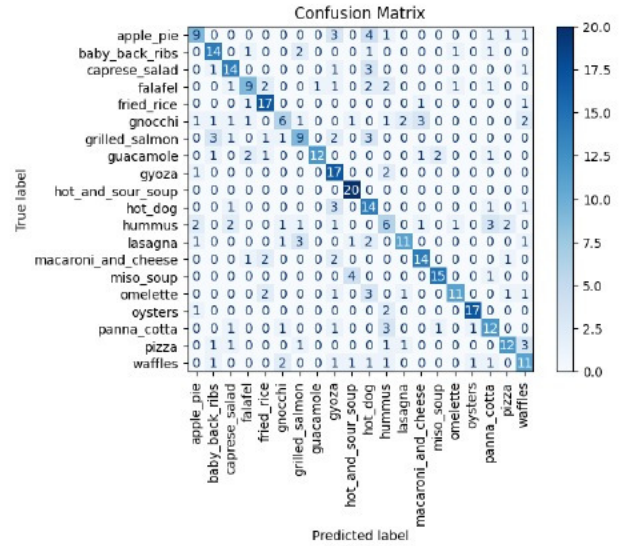


Figure 8: Run 6: Fine-tuned model confusion matrix.

## 5 Discussion

### 5.1 What We Observed

Early stopping helped us avoid wasting training time once the model stopped improving, but it didn't fundamentally change the overfitting pattern we observed in the base model. The model still memorized the training set.

Adding dropout ( $p = 0.5$ ) successfully reduced the train-validation gap, but surprisingly hurt test accuracy slightly. We suspect dropout may have been too aggressive for our already tiny dataset.

Weight decay had almost no measurable effect in our experiments, likely because our frozen ResNet encoder already provided strong regularization. However, we cannot rule out that weight decay might be more effective with different hyperparameter settings or with a deeper unfrozen network.

Data augmentation presented an interesting challenge. Aggressive augmentation (heavy rotations, color jitter, random erasing) reduced overfitting but also degraded accuracy, we appeared to be making the task harder without adding meaningful information. Switching to lighter augmentation (just crops and flips) struck what seemed like a better balance for our setup, giving us modest gains (1%) without hurting performance.

The most significant improvement in our experiments came from fine-tuning. Unfreezing the top three ResNet layers allowed the model to adapt ImageNet features to food-specific patterns, improving test accuracy from 55% to 62.5%. This suggests that pretrained features benefited from some domain adaptation. Letting the model adjust high-level representations while keeping low-level features frozen appeared to be effective for this dataset size and configuration.

## 5.2 Remaining Challenges

Even our best model struggled with certain classes. The confusion matrices revealed the model frequently confused visually similar dishes (like fried rice with macaroni and cheese) and struggled with classes that likely have high visual variability in our small sample (hummus, gnocchi). Grad-CAM visualizations showed that the model sometimes focused on what seemed like less relevant cues, such as plate edges, garnishes, background elements, rather than the food itself.

## 5.3 Reflections and Possible Directions

This project showed us that with our approach, transfer learning from ImageNet with standard regularization techniques, achieving high accuracy with very limited data proved difficult. Our 62.5% accuracy with 20 images per class falls well short of the 90% achievable with 1000 images per class, representing a substantial performance gap.

However, alternative approaches might yield better results in this low data scenario such as:

- **Few-shot learning:** Techniques specifically designed for extreme data scarcity (Prototypical Networks, MAML, meta-learning) might handle this scenario more effectively than our transfer learning approach.
- **Domain-specific pretraining:** Starting from a model pretrained on food images rather than general ImageNet could potentially provide better initial features.
- **Attention and localization:** Explicitly guiding the model to focus on the dish rather than background could help, though we're uncertain whether this would overcome the fundamental data limitation.
- **Data quality over quantity:** More careful curation of which 20 images to include, maximizing diversity while covering common variations, could yield better results with the scarce data.

Overall, this work illustrates that deep learning in severely data-limited environments remains challenging, at least with standard transfer learning approaches. While we achieved substantial improvements in terms of overfitting and accuracy, there is still a big gap compared to the model trained on the full Food-101 dataset.

## References

- [1] DanB et al. Food 101 Dataset. <https://www.kaggle.com/dansbecker/food-101> 2017. Original paper: *Food-101 – Mining Discriminative Components with Random Forests*.
- [2] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *Computer Vision – ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part VI*, pages 446–461. Springer, 2014. doi:[10.1007/978-3-319-10599-4\\_29](https://doi.org/10.1007/978-3-319-10599-4_29) URL [https://doi.org/10.1007/978-3-319-10599-4\\_29](https://doi.org/10.1007/978-3-319-10599-4_29)
- [3] Niki Martinel, Gian Luca Foresti, and Christian Micheloni. Wide-slice residual networks for food recognition. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 567–576, 2018.
- [4] Jake Snell, Kevin Swersky, and Richard S. Zemel. Prototypical networks for few-shot learning. *Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.